



Internship at CAVE LABS-PESU
"3D Evidence Reconstruction on android using 2D Videos"

Submitted by:

Gagana C	PES1UG24AM099
-----------------	----------------------

Under the guidance of

Dr. Adithya Balasubramanyam
Professor,

CAVE LABS-PESU

8 Weeks (June 2026 – July 2026)

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL
INTELLIGENCE AND MACHINE LEARNING)
FACULTY OF ENGINEERING**

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

DECLARATION

I, Gagana (PES1UG24AM099), hereby declare that the project titled "3D Evidence Reconstruction using 2D Videos" has been carried out at CAVE Labs by me under the guidance of Dr. Aditya Balasubramanyam, Project Guide, CAVE Labs, and submitted in partial fulfilment of the credits for the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) of PES University, Bengaluru, during the academic period June 2026 – July 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

Gagana C	PES1UG24AM099
-----------------	----------------------

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide, Dr. Adithya Balasubramanyam, CAVE LABS-PESU, for his continuous guidance, assistance and encouragement throughout the development of the SplatStudio project during my internship.

I am grateful to the internship coordinator, Prof. Indu Radhakrishnan, Department of CSE(AIML), PES University, for organizing, managing and helping out with the entire internship process.

I take this opportunity to thank Dr. Jayashree R, Chairperson, Department of CSE(AIML), PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor, PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing me various opportunities and enlightenment every step of the way.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents and my friends.

ABSTRACT

SplatStudio is an end-to-end pipeline that reconstructs a viewable 3D representation of a physical scene from an ordinary 2D smartphone video, built around the applied use case of 3D crime-scene evidence reconstruction. The overall system chains video capture, frame extraction (ffmpeg),

Structure-from-Motion pose recovery (COLMAP), Gaussian-Splat training (FastGS), and a statistical post-processing cleanup stage to produce a denoised, viewable 3D Gaussian Splat of the captured scene.

This report documents the work carried out by me over an 8-week internship at CAVE LABS-PESU, where my primary responsibility was the design and development of the Android client application that field users interact with directly. This included the camera capture module built on CameraX, an on-device Room database for job history and splat library persistence, a Retrofit/OkHttp/Moshi-based networking layer that uploads captured video and polls backend job status, an in-app 3D Gaussian Splat viewer built using a WebView and a custom Three.js-based shader renderer, the Jetpack Compose UI and configuration screens of the application, and a PowerShell-based Windows deployment script that mirrors the Linux setup tooling. I additionally contributed to an early AR Foundation-based capture prototype that was explored and later superseded by the native Android capture flow.

The report covers the architecture of the mobile application, the technologies and design decisions behind each module, how the application integrates with the backend processing pipeline, and the outcomes and learnings from the internship.

1.
1.1.

TABLE OF CONTENTS

	Introduction
	Overview
	1.2.Objectives of the Internship
	2. Company / Research Centre Introduction
	2.1.About CAVE LABS
	3. Internship Project Details
	3.1.Project Title
	3.2.Relation to a Larger Project
	3.3.My Role and Responsibilities
	4. Project Abstract and Scope
	5. Project Design Details and Technologies Used
	5.1.Overall System Architecture
	5.2.Mobile Application Architecture
	5.3.Technology Stack Summary
	6. Coding / Implementation Details
	6.1.Video Capture Module
	6.2.Local Persistence Module
	6.3.Backend Networking Module
	6.4.In-App 3D Splat Viewer Module
	6.5.UI / Configuration Module
	6.6.Windows Deployment Tooling
	6.7.Early AR Capture Prototype (Explored & Superseded)
	7. Project Results and Learning Outcomes
	7.1.Results
	7.2.Future Scope
	8. Conclusion
	References

LIST OF TABLES AND FIGURES

Figure 1: End-to-end 3dReConstruct pipeline

Figure 2: Mobile application module architecture

Table 1: Technology stack used in the mobile application

Table 2: Summary of modules implemented by the author

1. INTRODUCTION

1.1. Overview

Reconstructing a three-dimensional representation of a real-world scene from ordinary two-dimensional video has, until recently, required specialised hardware such as LiDAR scanners or multi-camera rigs. Advances in Structure-from-Motion (SfM), Neural Radiance Fields (NeRF), and more recently 3D Gaussian Splatting have made it possible to obtain high-fidelity 3D reconstructions from nothing more than a handheld video, using only classical geometry estimation and differentiable rendering. This shift is directly relevant to domains such as forensic documentation, where a scene must be preserved digitally, quickly, and without disturbing physical evidence. It is against this backdrop that the SplatStudio project was undertaken at CAVE LABS-PESU. The project applies this pipeline of video-to-3D-Gaussian-Splat reconstruction to a specific, applied use case: 3D crime-scene evidence reconstruction, where an investigator records a short handheld video of a scene, and the system produces a cleaned, inspectable 3D reconstruction that can be viewed, shared, and referred back to later.

1.2. Objectives of the Internship

- To gain hands-on experience with native Android development using Kotlin, Jetpack Compose, and CameraX-based video capture.
- To build an on-device data layer (Room) for reconstruction job history and a library of completed splats.
- To integrate the client with a FastAPI backend via a robust networking layer supporting multipart uploads and asynchronous job polling.
- To build an in-app 3D viewer for Gaussian Splat output, and understand the backend pipeline (COLMAP, FastGS, statistical cleanup) it depends on.

2. COMPANY / RESEARCH CENTRE INTRODUCTION

2.1. About CAVE LABS

CAVE (CAVE Augmented and Virtual Environments) Labs is a multidisciplinary research and innovation initiative at PES University, operating under the university's Centre for IoT. The lab works across Mixed Reality, Augmented and Virtual Reality, Digital Twins, human motion tracking, and related immersive-technology areas, providing a collaborative environment for students, researchers, and industry partners to explore and build next-generation immersive and applied computer-vision solutions, including uses in healthcare, remote assistance, and education.

The SplatStudio project undertaken during this internship sits within this broader applied-research charter: it takes the 3D Gaussian Splatting line of research — originally developed for photorealistic novel-view synthesis — and repurposes it for evidence-grade scene capture, where requirements such as auditability, robustness to poor lighting, and object-level isolation differ meaningfully from the original graphics use case. Work on this project spans the full stack: a native Android capture application, a Python/FastAPI backend that orchestrates video processing, third-party computer vision tools (ffmpeg, COLMAP), a Gaussian Splat training engine (FastGS), and a statistical post-processing layer that cleans the raw splat output before it is presented to the end user. Interns and contributors on the project take ownership of different layers of this stack; my own contribution, detailed in this report, was concentrated on the mobile application layer.



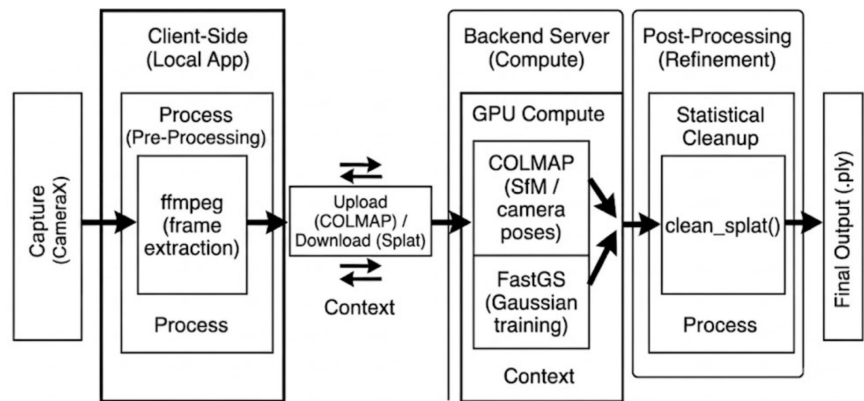
3. INTERNSHIP PROJECT DETAILS

3.1. Project Title

3D Evidence Reconstruction on android using 2D Videos

3.2. Relation to a Larger Project

This internship project is not a standalone academic exercise — it is one part of a larger, ongoing system being built at CAVE LABS. The complete system is an end-to-end pipeline that takes a phone video as input and produces a cleaned 3D Gaussian Splat as output, following the stage sequence:



The mobile application is the entry and exit point of this pipeline from a user's perspective: it is where the video is captured and uploaded, and where the final reconstruction is viewed. The backend stages (COLMAP, FastGS training, and the multi-stage statistical cleanup filter) were developed and owned by another contributor on the project; my role was to build the complete client-side application that captures the input and consumes the output of that backend pipeline, and to contribute to the shared repository infrastructure that ties the two together.

3.3. My Role and Responsibilities

Over the 8-week internship, my responsibilities centred on the Android mobile client of SplatStudio, spanning capture, on-device data management, networking, in-app 3D visualisation, UI, and cross-platform deployment tooling. Specifically, I was responsible for:

- Building the video capture screens and camera pipeline (MainActivity, RecordScreen) using CameraX.
- Designing the on-device persistence layer (Room database, DAO, repository, entities) for job history and the local splat library.
- Co-developing the networking layer (video upload, job-status polling) and the in-app 3D Splat viewer (WebView + custom Three.js renderer) with another team member.
- Building the application's UI/configuration screens (Jetpack Compose, Material theming), and writing the Windows PowerShell deployment script mirroring the team's Linux/macOS tooling.
- Exploring an early AR Foundation-based capture prototype, which informed the decision to ship the native Android capture flow instead.

4. PROJECT ABSTRACT AND SCOPE

The scope of the SplatStudio project, from the mobile application's point of view, is to provide a single, self-contained Android application that lets a field user (for example, an investigator documenting a scene) do three things: capture a video of a scene, send it for processing, and view the resulting 3D reconstruction — all without needing to understand or interact with the underlying computer vision pipeline.

The application scope, as implemented, includes: (a) a capture flow built on CameraX that records handheld video at a resolution and frame rate suitable for downstream Structure-from-Motion processing; (b) a local Room database that stores job history and a library of previously completed reconstructions, so the user can review past captures without a network round-trip; (c) a networking layer that performs a multipart upload of the captured video to the backend and polls a job-status endpoint until processing completes; (d) an in-app 3D viewer that renders the final Gaussian Splat directly on the device, without requiring a separate desktop viewer; and (e) supporting UI/configuration screens and cross-platform (Windows) developer tooling for the wider team.

Out of scope for the mobile application, and instead owned by the backend, are the actual reconstruction stages themselves — frame extraction, Structure-from-Motion, Gaussian Splat training, and the statistical cleanup filtering that removes noisy or spurious points from the raw splat before it is returned to the app. The mobile application treats the backend as a black box that accepts a video and eventually returns a cleaned `.ply` Gaussian Splat file.

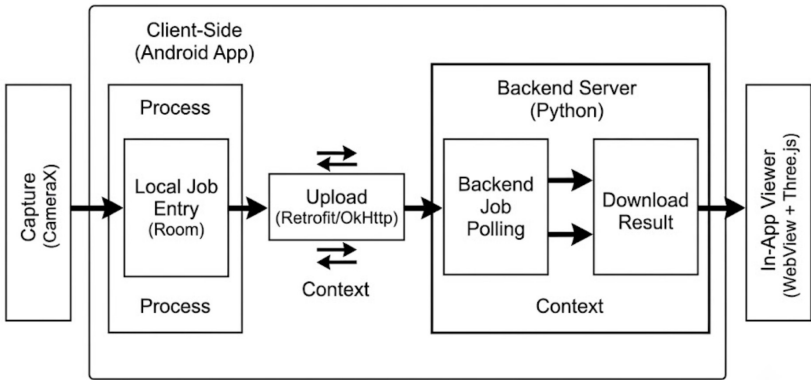
5. PROJECT DESIGN DETAILS AND TECHNOLOGIES USED

5.1. Overall System Architecture

At a system level, SplatStudio follows a client-server architecture. The Android client (my primary area of ownership) is responsible for capture, local job/library management, and visualisation. The FastAPI backend is responsible for orchestrating the reconstruction pipeline: extracting frames with ffmpeg, recovering camera poses and a sparse point cloud with COLMAP, training a 3D Gaussian Splat with FastGS, and running a multi-stage statistical cleanup filter (scale/aspect/dust filtering, opacity filtering, radial-outlier cropping, density-based clustering via DBSCAN, and optional RANSAC plane-consistency checks) before writing the final '.ply' file that the app later downloads and renders.

5.2. Mobile Application Architecture

The mobile application follows a layered architecture typical of modern Android apps: a UI layer built with Jetpack Compose, a data layer backed by a local Room database, and a networking layer built on Retrofit, OkHttp, and Moshi. Each screen (capture, library, splat viewer, configuration) is a Compose screen backed by its own state, with the Room repository acting as the single source of truth for job and library data, and the networking layer acting as the bridge to the backend.



5.3. Technology Stack Summary

Layer	Technology / Library	Purpose
Capture	CameraX (Kotlin)	Handheld video capture within MainActivity / RecordScreen
UI	Jetpack Compose, Material theming	Capture, library, viewer and configuration
Local persistence	Room (SQLite ORM)	On-device job history and splat library storage
Networking	Retrofit + OkHttp + Moshi	Multipart video upload, JSON job-status polling
In-app 3D viewer	WebView + Three.js (custom shader)	Rendering the Gaussian Splat result on-device
Deployment tooling	PowerShell (windows_setup.ps1)	Windows-side project setup, mirroring the Linux script
Prototype (explored)	Unity AR Foundation	Early capture prototype, later superseded

6. CODING / IMPLEMENTATION DETAILS

6.1. Video Capture Module

Files: *MainActivity.kt, RecordScreen.kt*

The capture module is the entry point of the application and the first stage of the overall pipeline. It uses CameraX to provide a stable, lifecycle-aware camera preview and video-recording surface, abstracting away the considerable device fragmentation that exists across Android camera hardware. RecordScreen presents the user with a live preview, start/stop recording controls, and basic capture-quality guidance, while MainActivity wires together the required runtime permissions (camera, microphone, storage) and hosts the Compose navigation graph.

A key design consideration in this module was matching the captured video's resolution and frame rate to what the backend pipeline actually needs: the backend samples frames at a fixed rate (6 fps) and a fixed working resolution (800×800) for COLMAP feature extraction, so the capture module was tuned to avoid recording unnecessarily high-bitrate video that would only be downsampled later, while still capturing enough visual detail and camera motion for COLMAP to recover reliable poses.

6.2. Local Persistence Module

Files: *SplatDatabase.kt, SplatDao.kt, SplatRepository.kt, Entities.kt*

Because reconstruction jobs can take a non-trivial amount of time on the backend (COLMAP feature matching and FastGS training are both computationally expensive), the application needed a reliable way to track jobs the user has submitted, and to build up a local library of completed reconstructions, independent of network availability. This module implements that using Room, Android's SQLite-backed ORM.

Entities.kt defines the on-device schema — job records (with fields such as job identifier, submission time, and status) and splat library entries (pointing to the locally cached result file once downloaded). SplatDao.kt exposes the query interface Room generates implementations for, and SplatRepository.kt wraps the DAO to present a single, coroutine-friendly API to the rest of the app, so that UI screens never talk to the database directly. This repository pattern kept the persistence

layer testable and decoupled from both the UI and the networking layer.

6.3. Backend Networking Module

Files: *NetworkClient.kt, SplatApiService.kt, Models.kt* (co-developed)

This module is the bridge between the mobile application and the FastAPI backend. *NetworkClient.kt* configures a shared *OkHttp* client and *Retrofit* instance, including timeouts long enough to tolerate large multipart video uploads over mobile networks. *SplatApiService.kt* defines the *Retrofit* interface used to submit a new reconstruction job (as a multipart upload of the captured video) and to poll ``/api/jobs/{id}`` for status updates, while *Models.kt* defines the Moshi-serialisable request/response data classes shared across both.

A deliberate design decision here was to model job status as a small state machine (queued, uploading, processing, done, failed) rather than a boolean flag, so the UI could show meaningful, granular progress to the user instead of a single indeterminate spinner. This also anticipated later backend work exposing more granular per-stage failure states (for example, distinguishing a COLMAP failure from a FastGS training failure) without requiring further changes on the client side.

6.4. In-App 3D Splat Viewer Module

Files: *SplatViewerScreen.kt, SplatHtmlTemplate.kt* (co-developed)

Once a reconstruction job completes and the cleaned ``ply`` Gaussian Splat file is downloaded, the user needs a way to actually view it — without this, the app would only be a capture-and-upload tool. Since there is no mature native Android library for rendering 3D Gaussian Splats, this module takes a hybrid approach: *SplatViewerScreen* hosts an Android *WebView*, and *SplatHtmlTemplate.kt* generates the HTML/JavaScript page loaded into that *WebView*, built around *Three.js*.

The core of this module is a custom ``GaussianSplatRenderer``, implemented as a *Three.js* *ShaderMaterial* of roughly 1,400 lines, which parses the raw Gaussian Splat point data (position, scale, rotation as a quaternion, opacity, and spherical-harmonic colour coefficients) and renders each Gaussian as a view-dependent, alpha-blended splat directly on the GPU, rather than as a naive point cloud. This gives users an interactive, rotate/pan/zoom 3D view of the reconstructed scene entirely on-device, without needing a desktop viewer or a separate export step.

6.5. UI / Configuration Module

Files: *ConfigScreen.kt, LibraryScreen.kt, and associated theming*

This module covers the remaining user-facing surface of the application. LibraryScreen presents the locally persisted history of jobs and completed reconstructions (backed by the persistence module described in section 6.2), letting the user revisit older splats without re-uploading anything. ConfigScreen exposes application-level settings, such as the backend endpoint being used. Both screens, along with the rest of the app, were built using Jetpack Compose with a consistent Material theme, keeping the visual language of the capture, library, viewer, and configuration screens unified.

6.6. Windows Deployment Tooling

File: *windows_setup.ps1*

While the backend's primary setup tooling targets Linux/macOS (with OS and package-manager detection, GPU/CUDA/RAM/disk compatibility checks, and isolated virtual environments for the backend and FastGS), several contributors on the project develop on Windows. To keep the project accessible across the team's development environments, I wrote *windows_setup.ps1*, a PowerShell equivalent of the Linux/macOS setup script, so that Windows-based contributors could set up the backend and its dependencies without needing a Linux environment or WSL.

6.7. Early AR Capture Prototype (Explored and Superseded)

Before settling on the native CameraX-based flow, I contributed to an early Unity AR Foundation-based capture prototype, alongside other team members, aimed at giving real-time camera-coverage feedback during filming. It was not carried forward, since it added Unity as a second, heavyweight mobile runtime without a matching gain in reliability, and the team judged the native Android flow simpler to maintain. The coverage-feedback idea remains a possible future enhancement.

Figure 1: SplatStudio Android

App – Capture Screen

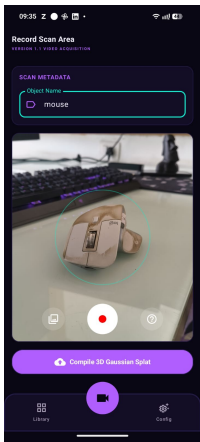


Figure 2: SplatStudio Android App – Progress Bar

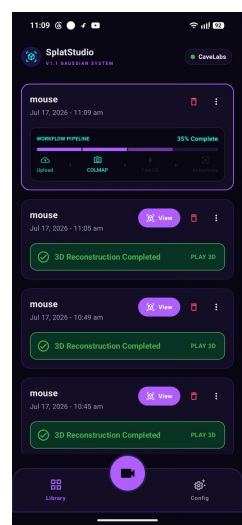


Figure 3: SplatStudio Android App – Rendered View



Figure 4: SplatStudio Android App – Audit Log

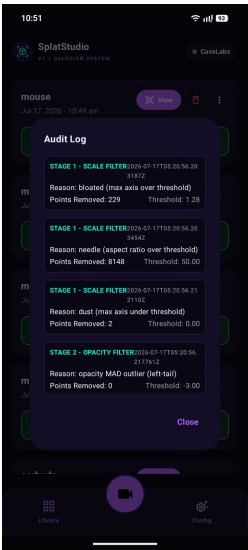


Figure 5: SplatStudio Android

App – Metrics

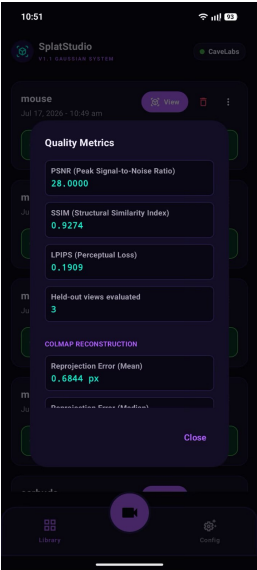
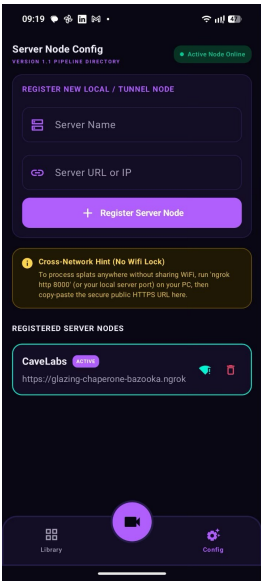


Figure 6: SplatStudio Android App – Server page



7.PROJECT RESULTS AND LEARNING OUTCOMES

7.1.Results

By the end of the 8-week internship, the mobile application supported a complete, working user flow: recording a video of a scene, uploading it to the backend, tracking its processing status, downloading the resulting cleaned Gaussian Splat, and viewing it interactively in-app, with a persistent local library of past captures. This closed the loop on the client side of the 3dReConstruct pipeline and gave the wider team a usable interface to the backend reconstruction system described in section 3.2, rather than requiring manual API calls or a desktop viewer to see results.

A summary of the modules delivered as part of this internship is given below.

#	Module	Key Outcome
1	Video Capture	Reliable CameraX-based capture flow tuned to backend frame-rate/resolution requirements
2	Local Persistence	Room-backed job history and splat library, available offline
3	Backend Networking	Multipart upload + granular job-status polling against /api/jobs/{id}
4	In-App 3D Viewer	Custom Three.js Gaussian Splat shader renderer inside a WebView
5	UI / Configuration	Consistent Compose-based UI across capture, library and settings screens
6	Windows Tooling	PowerShell setup script enabling Windows-based development

Beyond the concrete deliverables, this internship was a significant learning experience. On the technical side, I gained practical experience with modern Android architecture (Jetpack Compose, Room), asynchronous networking patterns (Retrofit/OkHttp/Moshi, including long-running job polling), and, notably, applying WebGL/Three.js shader concepts to Gaussian Splat rendering — well outside typical mobile-app rendering work — which required understanding the underlying Gaussian Splat data representation (position, scale, rotation quaternion, opacity, spherical-harmonic colour) well enough to parse and render it correctly. I also gained a working understanding of the backend pipeline the app depends on (ffmpeg → COLMAP → FastGS → statistical cleanup), which was essential context for client-side decisions such as capture resolution/frame-rate and job-status structuring, even though I did not own that half of the codebase myself.

On the collaborative side, working within an existing, actively developed codebase required more

coordination than a typical academic assignment: several modules (networking) were built jointly with another team member, which meant agreeing on interfaces and data contracts up front, and the Windows setup script had to stay functionally equivalent to an existing, giving me practical experience with the interface discipline expected in a real engineering team.

7.2. Future Scope

While the foundational pipeline is functionally robust, there are several avenues for future enhancement to elevate the system to enterprise-grade forensic standards:

- **Semantic Object Segmentation:** While current object isolation relies on spatial heuristics (RANSAC and DBSCAN), future iterations could integrate lightweight visual foundation models (e.g., Segment Anything) to provide semantic, object-aware tagging of specific evidence without requiring manual threshold tuning.
- **Dynamic Threshold Optimization:** The adaptive MAD-based filtering could be further augmented with a machine-learning feedback loop, allowing the backend to dynamically adjust geometric outlier thresholds based on lighting and texture conditions extracted from the initial video frames.
- **Enhanced On-Device Rendering:** Further optimization of the Android viewer is planned, including implementing level-of-detail (LOD) degradation and tiled rendering, to support smooth navigation of dense splats on lower-end mobile devices.

8. CONCLUSION

This internship at CAVE LABS-PESU gave me the opportunity to build the complete mobile client of an applied 3D reconstruction system, SplatStudio, end to end — from video capture through to on-device visualisation of the final Gaussian Splat result. Over the 8-week period, I designed and implemented the capture flow, the local persistence layer, contributed to the backend networking and in-app 3D viewer, built the application's UI and configuration screens, and wrote the Windows-side deployment tooling for the project.

Working on a live, multi-contributor codebase rather than an isolated assignment meant engaging directly with the constraints of a real system: matching client-side capture parameters to the backend's actual processing requirements, agreeing on shared interfaces with other contributors, and building a viewer capable of correctly rendering a non-trivial 3D data format (Gaussian Splats) that most standard mobile libraries do not support out of the box. The result is a mobile application that meaningfully lowers the barrier to using the underlying reconstruction pipeline, letting a user go from a handheld video to an interactive 3D scene view entirely through the app, without manual intervention at any intermediate stage.

Overall, the internship strengthened my practical skills in native Android development, client-server system design, and applied 3D graphics programming, while also giving me exposure to the broader Structure-from-Motion and Gaussian Splatting techniques that underpin the project's backend, and to the practical realities of contributing to a shared, evolving codebase as part of a small engineering team.

REFERENCES

1. B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
2. J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
3. Android Developers, "CameraX Overview," developer.android.com/training/camerax, accessed 2026.
4. Android Developers, "Room Persistence Library," developer.android.com/training/data-storage/room, accessed 2026.
5. Square Inc., "Retrofit — A type-safe HTTP client for Android and Java," square.github.io/retrofit, accessed 2026.
6. Three.js Documentation, threejs.org/docs, accessed 2026.
7. CAVE Labs internal repository, 3dReConstruct, github.com/adityandandia/3dReConstruct (internal/private repository).